

Instituto por Cooperativa Villa Hermosa

Curso: Programación II

FASE 3

DESARROLLO / CÓDIGO

Nombre: Yuli Gabriela Villagrán Ramírez

Grado: 5to. PC

Sección: A

Clave: 33

```
import os
import random
```

```
# =====
# ETAPA 3: DESARROLLO DEL CÓDIGO
# =====
```

```
"""
=====
```

CLASE PADRE (SUPERCLASE): ElementoJuego

Representa cualquier entidad u objeto base que existe dentro del videojuego.
Define los atributos comunes (nombre, emoji o símbolo y coordenadas X, Y)
y el método base 'mostrar_info()' que será sobrescrito por sus clases hijas
para aplicar POLIMORFISMO.

```
=====
"""
```

```
class ElementoJuego:
```

```
    def __init__(self, nombre, simbolo, x=0, y=0):
        self.nombre = nombre
        self.simbolo = simbolo
        self.x = x
        self.y = y
```

```
    # Método base para el polimorfismo
```

```
    def mostrar_info(self):
        return f"Entidad: {self.nombre} | Símbolo: {self.simbolo} | Posición: ({self.x}, {self.y})"
```

```
"""
=====
```

CLASE HIJA 1 (SUBCLASE): Personaje (Hereda de ElementoJuego)

Representa al héroe controlado por el jugador.
Hereda el nombre, símbolo () y posición, y agrega atributos como
salud (corazones), energía, monedas e inventario (mochila).

```
=====
"""
```

```
class Personaje(ElementoJuego):
```

```
    def __init__(self, nombre):
        super().__init__(nombre, "", x=0, y=0)
        self.salud = 3    # 3 corazones de vida
        self.energia = 25 # 25 puntos de energía para el Nivel 1
        self.monedas = 0  # Contador de monedas
        self.mochila = [] # Inventario de herramientas
```

```
    # OCURRE EL POLIMORFISMO: Se rediseña mostrar_info() para mostrar el HUD del jugador
```

```
    def mostrar_info(self):
        corazones = " " * self.salud
        return f"Jugador: {self.nombre} {self.simbolo} | Vidas: {corazones}({self.salud}) | Energía: {self.energia} |  
Monedas: {self.monedas} 
```

```
"""
=====
```

CLASE HIJA 2 (SUBCLASE): Enemigo (Hereda de ElementoJuego)

Representa las amenazas del juego como los Glitches o el Rey Glitch.
Hereda la estructura base de entidad e incorpora el nivel de daño.

```
=====
"""
class Enemigo(ElementoJuego):
    def __init__(self, nombre, dano=1, x=0, y=0):
        super().__init__(nombre, "👾", x, y)
        self.dano = dano

    # OCURRE EL POLIMORFISMO: Muestra los datos de la amenaza
    def mostrar_info(self):
        return f"¡AMENAZA! {self.nombre} {self.simbolo} | Daño: {self.dano} corazón(es)"

=====
```

CLASE HIJA 3 (SUBCLASE): ObjetoInteractuable (Hereda de ElementoJuego)

Representa los elementos recolectables o destructibles (madera, rocas, objetos).

```
=====
"""
class ObjetoInteractuable(ElementoJuego):
    def __init__(self, nombre, simbolo, funcion, x=0, y=0):
        super().__init__(nombre, simbolo, x, y)
        self.funcion = funcion

    # OCURRE EL POLIMORFISMO: Describe la utilidad o función del objeto
    def mostrar_info(self):
        return f"Objeto: {self.nombre} {self.simbolo} - Descripción: {self.funcion}"

=====
```

```
# =====
# FUNCIONES AUXILIARES Y MANEJO DE PANTALLA
# =====
```

```
def limpiar_pantalla():
    """Limpia la terminal según el sistema operativo (Windows/Linux/Mac)."""
    os.system('cls' if os.name == 'nt' else 'clear')
```

```
def pausar():
    """Pausa la ejecución hasta que el usuario presione Enter."""
    input("\nPresiona [ENTER] para continuar...")
```

```
# =====
# ETAPA 2: FLUJO DE CADA OPCIÓN DEL MENÚ (LÓGICA Y NIVELES)
# =====
```

```
def nivel_1_bosque(jugador):
    """
    NIVEL 1: El Bosque de Píxeles (Exploración y Recursos)
    - Matriz 6x6
    - Energía inicial = 25 (resta 1 por paso)
    - Objetivo: Juntar 3 maderas (🌳)
```

- Regla especial: 5 monedas restauran la vida al máximo (3 ❤️)

"""

limpiar_pantalla()

print("=====")

print(" NIVEL 1: EL BOSQUE DE PÍXELES (Mundo 1 - Matriz 6x6) ")

print("=====")

print("Objetivo: Recolecta 3 maderas (🌲) antes de quedarte sin energía.\n")

maderas_recolectadas = 0

jugador.energia = 25 # Reiniciar energía para el nivel

Instanciamos herramientas y enemigos con Polimorfismo

pico = ObjetoInteractuable("Pico de Datos", "🔨", "Destruye rocas (🌲) que bloquean el camino")

glitch = Enemigo("Glitch de Código", dano=1)

Ciclo principal del Nivel 1

while maderas_recolectadas < 3 and jugador.energia > 0 and jugador.salud > 0:

limpiar_pantalla()

print("=====")

print(" NIVEL 1: EN CURSO ")

print("=====")

Demostración del polimorfismo imprimiendo el HUD

print(jugador.mostrar_info())

print(f"Progreso Maderas: {maderas_recolectadas}/3 🌲")

print("-----")

print("Controles: [W] Arriba | [S] Abajo | [A] Izquierda | [D] Derecha")

print(" [E] Abrir Mochila | [Q] Pedir Pista (2 🌲)")

print("-----")

Manejo de errores try-except para los controles de usuario

try:

paso = input("Ingresa tu movimiento: ").strip().lower() # Tolerancia a mayúsculas

if paso not in ['w', 'a', 's', 'd', 'e', 'q']:

raise ValueError("Tecla invalida")

except ValueError:

print(" ⚠️ Error: Usa únicamente las teclas W, A, S, D, E o Q.")

pausar()

continue

Uso de Mochila / Herramientas (Tecla E)

if paso == 'e':

print(f"\n🎒 MOCHILA:")

print(pico.mostrar_info())

pausar()

continue

Pista (Tecla Q)

if paso == 'q':

if jugador.monedas >= 2:

jugador.monedas -= 2

print("\n💡 PISTA: Muévete hacia los bordes para encontrar maderas ocultas.")

else:

```
    print("\n ⚠ No tienes suficientes monedas para solicitar una pista (Cuesta 2 🪙).")
    pausar()
    continue
```

```
# Restar 1 de energía por paso efectuado
jugador.energia -= 1
```

```
# Eventos aleatorios del escenario
evento = random.randint(1, 10)
```

```
if evento in [1, 2, 3]:
    maderas_recolectadas += 1
    print("\n🪵 ¡Encontraste y recolectaste 1 bloque de madera!")
elif evento in [4, 5]:
    jugador.monedas += 5
    print("\n📦 ¡Encontraste un cofre con 5 Monedas!")
    # Requerimiento: Juntar 5 monedas restaura la salud a 3 corazones
    if jugador.salud < 3:
        jugador.salud = 3
        print("💖 ¡Tus 5 monedas han restaurado tu salud al máximo (3 💖)!")
elif evento == 6:
    print(f"\n{glitch.mostrar_info()}")
    if "Escudo Protector" in jugador.mochila:
        print("🛡️ ¡Tu Escudo Protector absorbió el ataque del Glitch y se rompió!")
        jugador.mochila.remove("Escudo Protector")
    else:
        jugador.salud -= 1
        print("💣 ¡El Glitch te atacó! Perdiste 1 corazón de vida 💖.")
```

```
    pausar()
```

```
# Evaluación de fin de nivel
if jugador.salud <= 0 or jugador.energia <= 0:
    print("\n❌ Nivel Fallido: Te has quedado sin energía o vidas.")
    return False
```

```
print("\n🎉 ¡NIVEL 1 COMPLETADO! Maderas recolectadas. ¡Mapa Mundo 2 Desbloqueado!")
pausar()
return True
```

```
def nivel_2_carrera(jugador):
```

```
    """
    NIVEL 2: La Carrera de los Números (Obstáculos y Acertijo)
    - Obstáculo en la pista
    - Reto matemático / secuencia lógica
    """
```

```
    limpiar_pantalla()
    print("=====")
    print(" NIVEL 2: LA CARRERA DE LOS NÚMEROS (Mundo 2) ")
    print("=====")
    print("Avanzando por la pista lineal... 🏃🏻➡️")
    print("🚧 ¡PELIGRO! Un obstáculo bloquea el camino.")
```

```

print("Para saltar la tubería, resuelve la secuencia lógica:")
print("\n -> 3, 6, 12, ¿...? <- \n")

# Manejo de errores try-except para validar la respuesta numérica
try:
    respuesta = int(input("Introduce el número correcto para saltar: ").strip())
    if respuesta == 24:
        print("\n  ¡CORRECTO! El patrón se duplica (12 x 2 = 24).")
        print("\n  ¡Saltaste el obstáculo con éxito y cruzaste la meta!")
        pausar()
        return True
    else:
        print("\n  Respuesta incorrecta. Chocaste contra la tubería y perdiste el nivel.")
        jugador.salud -= 1
        pausar()
        return False
except ValueError:
    print("\n  Error: Debes ingresar un valor numérico entero.")
    jugador.salud -= 1
    pausar()
    return False

def nivel_3_fortaleza(jugador):
    """
    NIVEL 3: El Núcleo de la Fortaleza (Examen Final de Lógica)
    - 3 Preguntas lógicas consecutivas
    - Si acierta las 3: Despliega el trofeo ASCII de victoria
    """

    limpiar_pantalla()
    print("=====")
    print(" NIVEL 3: EL NÚCLEO DE LA FORTALEZA (Mundo 3 - Jefe Final)")
    print("=====")
    print("\n  El Rey Glitch bloquea la salida con 3 candados lógicos.")
    print("Debes responder correctamente las 3 preguntas consecutivas.\n")

    preguntas = [
        ("1. ¿Qué estructura de control se utiliza para repetir un bloque de código mientras una condición sea verdadera?\n 1) if-else\n 2) while\n 3) try-except", 2),
        ("2. ¿Qué bloque de código se utiliza en Python para capturar y manejar errores previniendo cierres inesperados?\n 1) try-except\n 2) for-in\n 3) def", 1),
        ("3. ¿Cuál es la palabra clave para definir una función en Python?\n 1) class\n 2) function\n 3) def", 3)
    ]

    aciertos = 0

    # Ciclo for para evaluar las preguntas
    for p_texto, r_correcta in preguntas:
        print("-----")
        print(p_texto)
        try:
            opcion = int(input("\nTu respuesta (1-3): ").strip())
            if opcion == r_correcta:

```

```

    print("✅ ¡Candado desbloqueado!")
    aciertos += 1
else:
    print("❌ Respuesta incorrecta. El candado se atascó.")
except ValueError:
    print("⚠️ Entrada no válida. Contada como respuesta incorrecta.")

```

Verificación final de victoria

```

if aciertos == 3:
    limpiar_pantalla()
    print("=====")
    print("🏆 ¡FELICITACIONES! ¡HAS GANADO! 🏆 ")
    print("=====")
    print(f"¡{jugador.nombre}, ERES EL MAESTRO SUPREMO DE LA LÓGICA! ")
    print("""
        _____
        | _ _ _ _ _ |
        | .\\:  /-. |
        | (|:  |) | |
        | -|:  |- ' |
        | \\::  /   |
        | ':.  '   |
        | ) (      |
        | -.' _ _   |
        | _ _ _ _ _ |
    """)
    print("=====")
    pausar()
    return True
else:
    print(f"\n❌ Has fallado. Acertaste {aciertos}/3 preguntas. La Fortaleza sigue cerrada.")
    pausar()
    return False

```

def flujo_jugar(jugador):

"""Gestión secuencial de los 3 niveles del Diagrama de Flujo"""

Nivel 1

if nivel_1_bosque(jugador):

Nivel 2

if nivel_2_carrera(jugador):

Nivel 3

nivel_3_fortaleza(jugador)

def ver_inventario(jugador):

"""Opción: Inventario y Vestuario (Herramientas)"""

limpiar_pantalla()

print("=====")

print("INVENTARIO Y VESTUARIO (MOCHILA)")

print("=====")

print("1. Equipar Traje 1 (Pico de Datos) 🛠️")

print("2. Equipar Traje 2 (Escudo Protector) 🛡️")

print("3. Equipar Traje 3 (Pociones de Energía) 🧪")

```
print("4. Volver al Menú Principal")
```

```
try:
```

```
opc = int(input("\nSelecciona una opción (1-4): ").strip())
```

```
if opc == 1:
```

```
    if "Pico de Datos" not in jugador.mochila:
```

```
        jugador.mochila.append("Pico de Datos")
```

```
        print("✅ ¡Pico de Datos equipado en la mochila!")
```

```
    else:
```

```
        print("❗ Ya tienes el Pico de Datos equipado.")
```

```
elif opc == 2:
```

```
    if "Escudo Protector" not in jugador.mochila:
```

```
        jugador.mochila.append("Escudo Protector")
```

```
        print("✅ ¡Escudo Protector equipado en la mochila!")
```

```
    else:
```

```
        print("❗ Ya tienes el Escudo Protector equipado.")
```

```
elif opc == 3:
```

```
    jugador.energia += 10
```

```
    print("🧴 ¡Poción consumida! Restauraste +10 puntos de energía.")
```

```
elif opc == 4:
```

```
    return
```

```
else:
```

```
    print("⚠️ Opción fuera de rango.")
```

```
except ValueError:
```

```
    print("⚠️ Error: Ingresa un número entero.")
```

```
    pausar()
```

```
def ver_opciones():
```

```
    """Opción: Configuración del juego"""
```

```
    limpiar_pantalla()
```

```
    print("=====")
```

```
    print("          OPCIONES          ")
```

```
    print("=====")
```

```
    print("🔊 Sonido: [Efectos: ACTIVADOS] | [Música: ACTIVADA]")
```

```
    print("📧 Soporte: Correo de asistencia activo")
```

```
    print("🔔 Notificaciones: Activadas")
```

```
    print("📖 Créditos: Desarrollado para Programación II")
```

```
    print("🎮 Legal: Videojuego Educativo en Consola Python")
```

```
    pausar()
```

```
def modo_multijugador():
```

```
    """Opción: Modo 3 Jugadores"""
```

```
    limpiar_pantalla()
```

```
    print("=====")
```

```
    print("          MODO 3 JUGADORES          ")
```

```
    print("=====")
```

```
    print("¿Primero en ganar 3 niveles?\n")
```

```
    jugadores = []
```

```
    for i in range(1, 4):
```

```
        nombre = input(f"Ingrese el nombre del Jugador {i}: ").strip() or f"Jugador_{i}"
```

```

jugadores.append(Personaje(nombre))

print("\n🎮 Jugadores Registrados en la Sesión:")
for j in jugadores:
    print(f" -> {j.mostrar_info()}") # Aplicación de Polimorfismo

pausar()

def ver_referencias():
    """Opción: Tabla de Referencias"""
    limpiar_pantalla()
    print("=====")
    print("      TABLA DE REFERENCIAS (TECLAS Y EMOJIS)      ")
    print("=====")
    print("👤 : Personaje principal (Vector)")
    print("🧑‍🦹 : Enemigo Glitch / Rey Glitch")
    print("🪵 : Madera (Recurso)")
    print("🪨 : Roca (Obstáculo)")
    print("💰 : Monedas (5 Monedas = Salud Máxima 3 ❤️)")
    print("\nControles de Consola: [W, A, S, D] Movimiento | [E] Mochila")
    pausar()

# =====
# ETAPA 1: BIENVENIDA Y MENÚ PRINCIPAL (ESTRUCTURA BASE)
# =====

def menu_principal():
    """Bienvenida y Menú Principal con estructura a prueba de fallos"""
    limpiar_pantalla()
    print("=====")
    print(" ¡BIENVENIDO A EL CAMINO DEL CÓDIGO: LA FORTALEZA DE LOS GLITCHES! ")
    print("=====")
    print("      Cargando juego...      ")
    print("=====")

    nombre_jugador = input("\n¿Cómo te llamas, explorador?: ").strip() or "Vector"
    jugador = Personaje(nombre_jugador)

    # Ciclo Principal (While True) del Menú
    while True:
        limpiar_pantalla()
        print("=====")
        print("      MENÚ PRINCIPAL      ")
        print("=====")
        print(f" Usuario activo: {jugador.nombre}")
        print("-----")
        print(" 1. Jugar")
        print(" 2. Inventario / Mochila")
        print(" 3. Opciones")
        print(" 4. Modo 3 Jugadores")
        print(" 5. Tabla de Referencias")
        print(" 6. Salir")
        print("=====")

```

Captura y prevención de errores mediante try-except

try:

```
opcion = int(input("Selecciona una opción (1-6): ").strip())
```

```
if opcion == 1:
```

```
    flujo_jugar(jugador)
```

```
elif opcion == 2:
```

```
    ver_inventario(jugador)
```

```
elif opcion == 3:
```

```
    ver_opciones()
```

```
elif opcion == 4:
```

```
    modo_multijugador()
```

```
elif opcion == 5:
```

```
    ver_referencias()
```

```
elif opcion == 6:
```

```
    # Submenú confirmación de salida
```

```
    confirmar = input("\n¿Está seguro de que desea salir? (s/n): ").strip().lower()
```

```
    if confirmar == 's':
```

```
        print("\n¡Gracias por jugar! ¡Hasta la próxima! 🎮")
```

```
        break
```

```
else:
```

```
    print("\n ⚠️ Opción no válida. Ingresa un número entre 1 y 6.")
```

```
    pausar()
```

except ValueError:

```
    print("\n ⚠️ Error: Ingresa únicamente un número entero.")
```

```
    pausar()
```

Punto de entrada para la ejecución del programa

```
if __name__ == "__main__":
```

```
    menu_principal()
```